Internet - of - Things

cheap

A Beginners Guide to the Internet and ESP8266



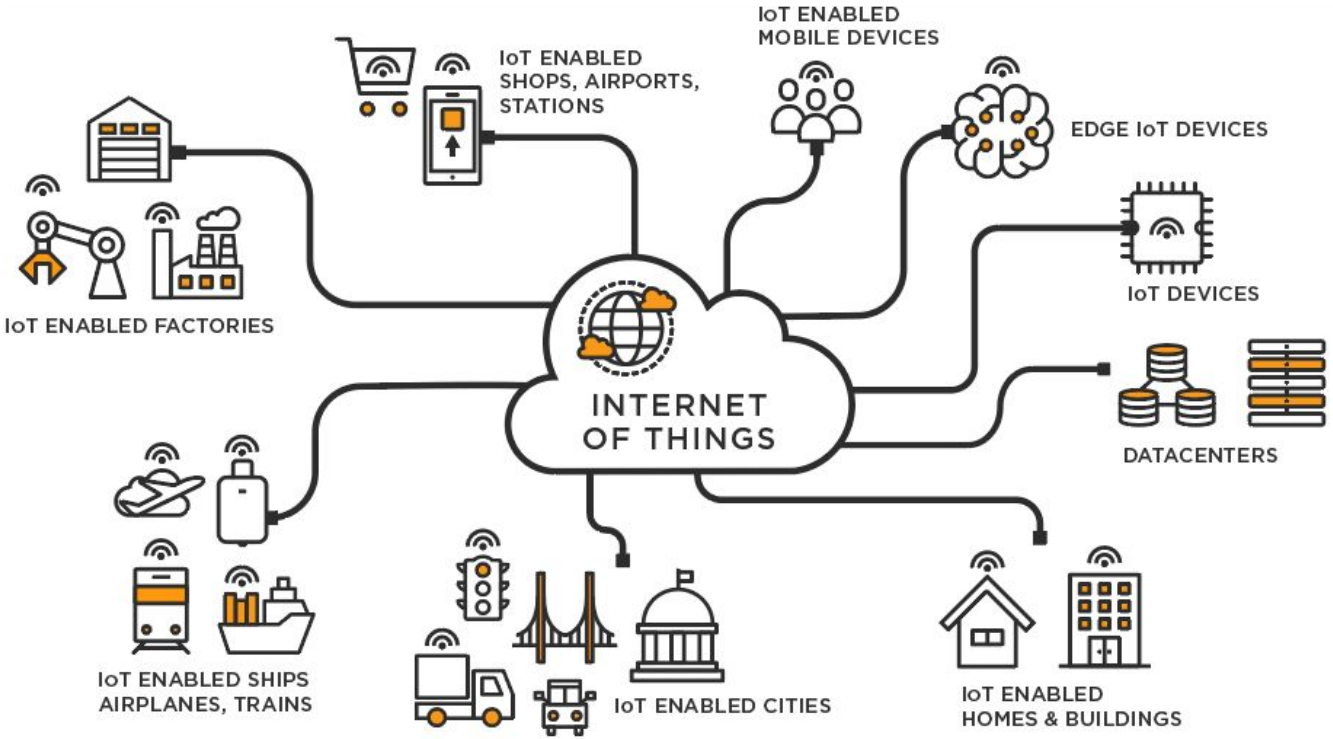
SINF - Semana de Informática, 2021
FEUP, Porto
João Pedro Dias & Bruno Lima

IoT, WoT, IoE, CPS, ...

“The network of devices that contain the hardware, software, firmware, and actuators which allow the devices to connect, interact, and freely exchange data and information.”

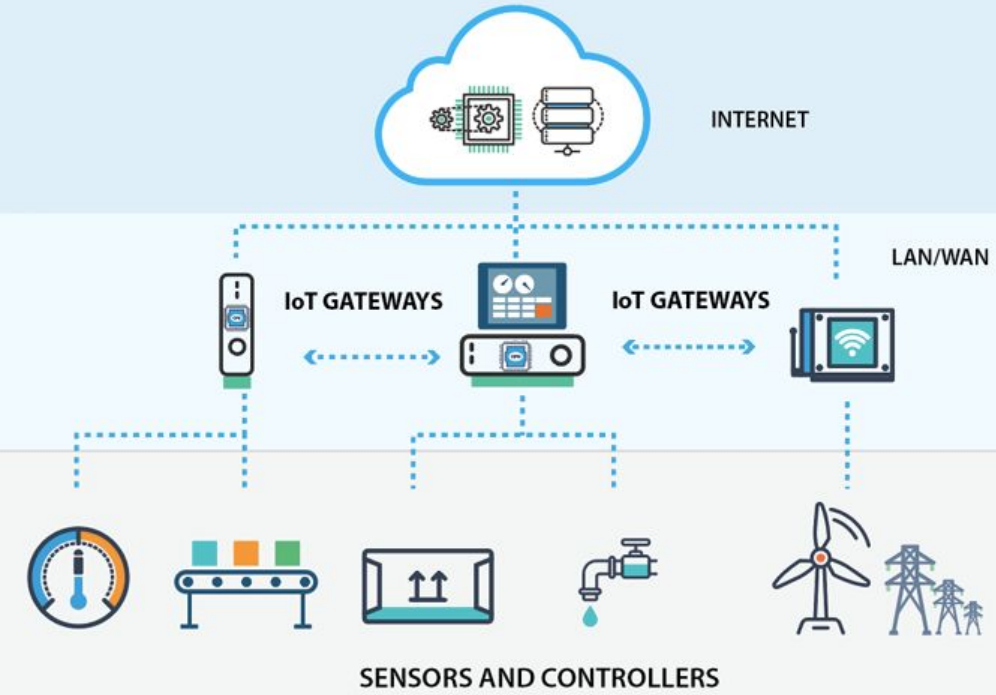
“(...) user or industrial devices that are connected to the internet. IoT devices include sensors, controllers, and household appliances.”

Internet of Things (IoT), NIST, https://csrc.nist.gov/glossary/term/internet_of_things_iot



What is the Internet of Things (IoT)?, TIBCO Software, https://www.tibco.com/reference-center/what-is-the-internet-of-things-iot

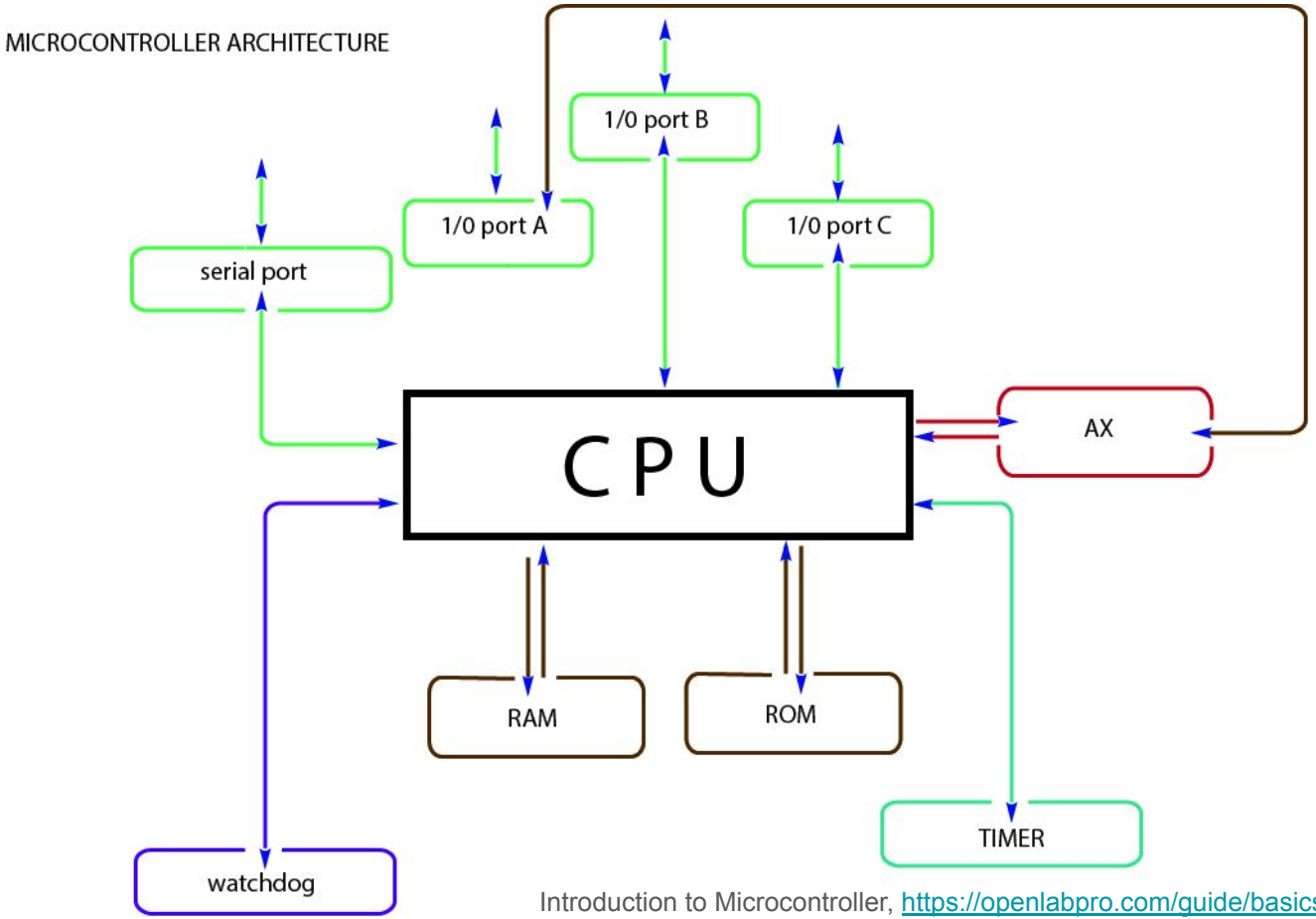
The IoT Three Tiers



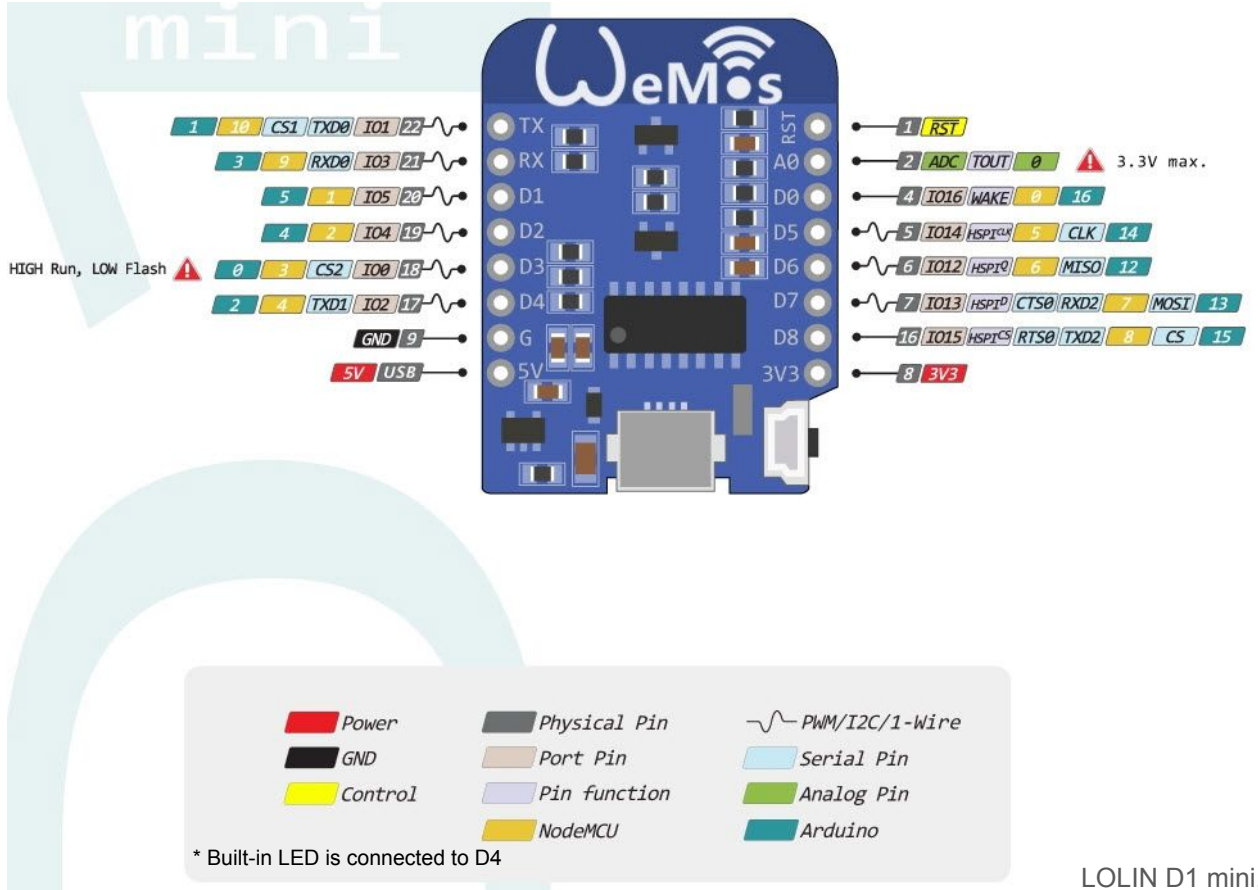
Cloud Tier: (Virtualized) High-power Servers and Services

Fog Tier: Gateways, Data Aggregators, Pre-Processing, etc.

Edge Tier: Sensors, Actuators, and other Low-computational Tasks



Introduction to Microcontroller, https://openlabpro.com/guide/basics-of-microcontroller/



ESP8266

(Wemos D1 mini)

Operating Voltage	3.3V
Digital I/O Pins	11
Analog Input Pins	1(3.2V Max)
Clock Speed	80/160MHz
Flash	4Mb
RAM	80Kb
Connectivity	Wi-Fi
Cost	2-3$

LOLIN D1 mini, https://www.wemos.cc/en/latest/d1/d1_mini.html

Actuator (LED)

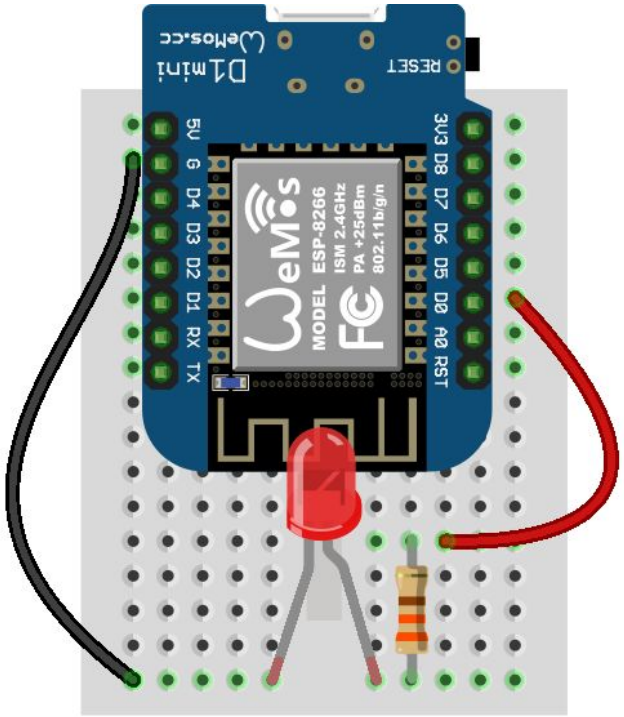
Actuators can be turned on/off by toggling a pin (e.g., **D0**). Other control modes exist, e.g., controlling a motor or the brightness of a LED can require pulse-width modulation (PWM).

In Arduino language, **`digitalWrite()`**:

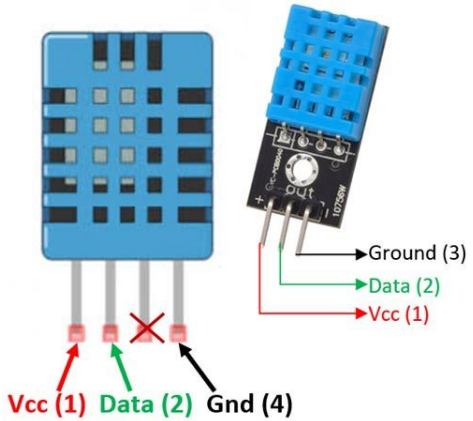
If the pin has been configured as an **`OUTPUT` with `pinMode()`**, its voltage will be set to the corresponding value:

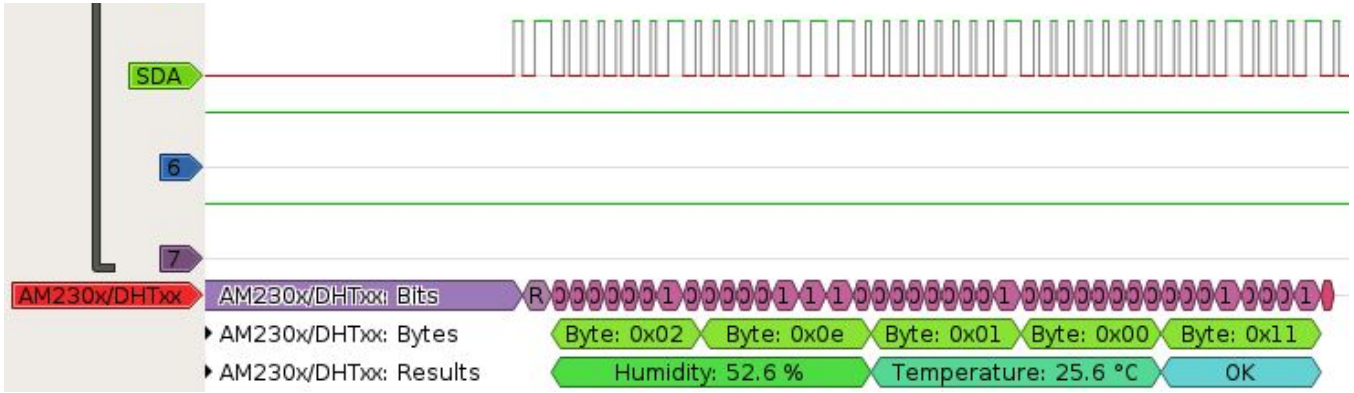
- **5V (or 3.3V on 3.3V boards) for `HIGH`**
- **0V (ground) for `LOW`**

`Wemos D1 mini has a built-in LED (part of the ESP8266 MCU), used for signalling RX/TX activity, but can be used for other purposes.`



Sensors (DHT11)

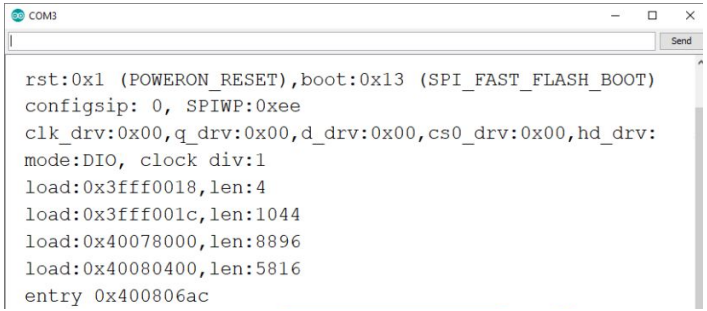




DHT11 is a **single wire digital humidity and temperature sensor**, which provides humidity and temperature values serially with **one-wire protocol**. DHT11 sensor provides relative **humidity value in percentage (20 to 90% RH)** and **temperature values in degree Celsius (0 to 50 °C)**.

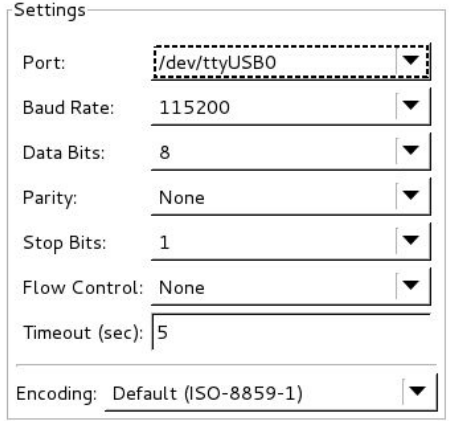
DHT11, https://www.electronicwings.com/sensors-modules/dht11

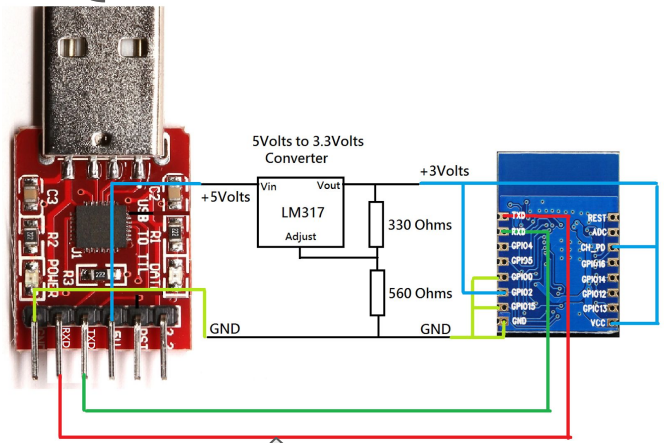
MCU <-> UART <-> USB <-> Terminal



Serial Terminal

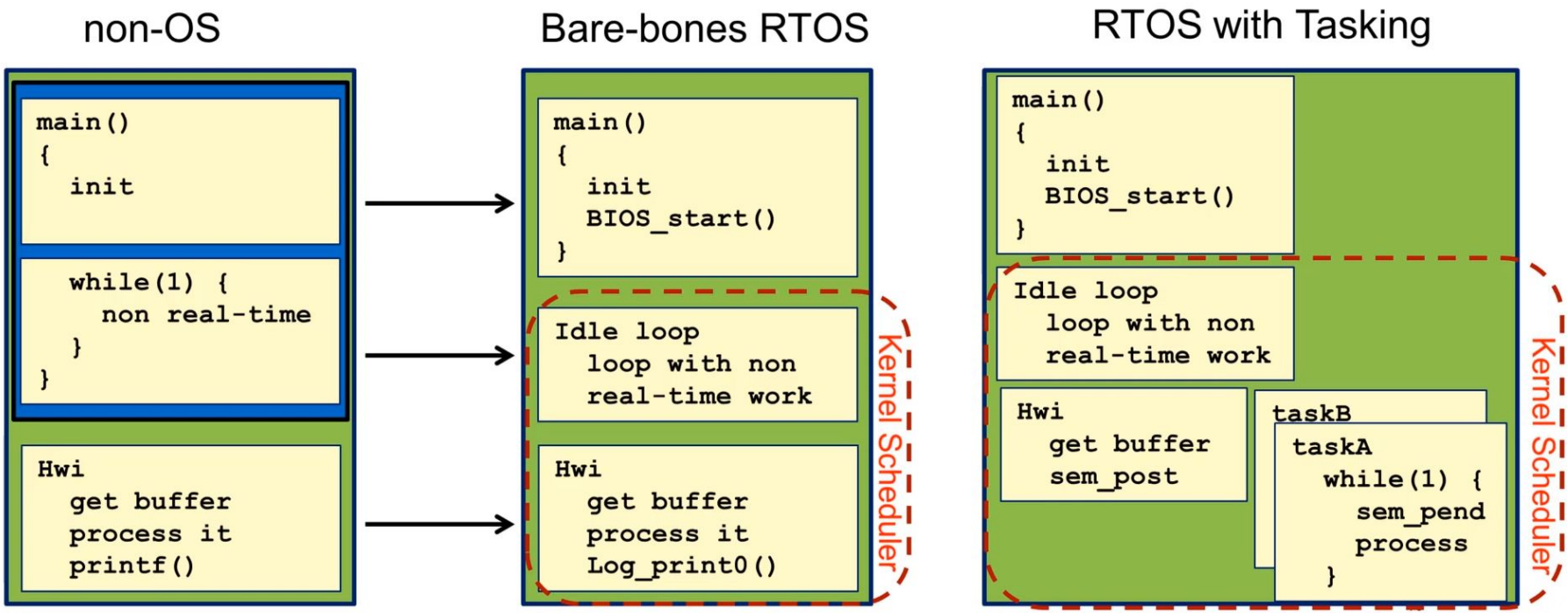
Serial Port Configuration





UART to USB Adapter
(built-in in Wemos D1 and most dev boards)

The many faces of Programming Embedded Devices

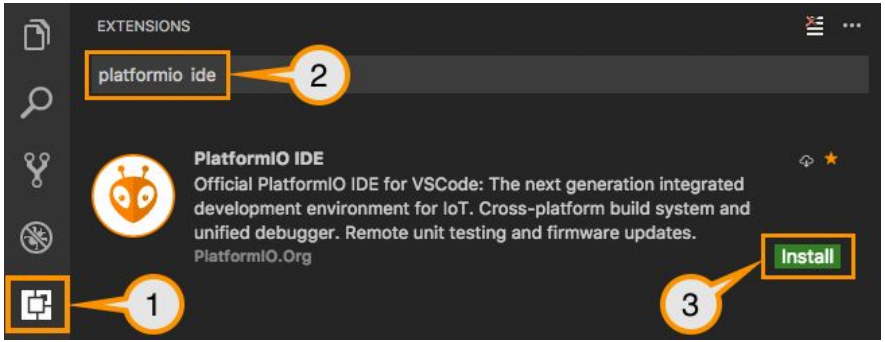


RTOS Concepts overview, https://training.ti.com/rtos-concepts-overview?context=1128562-1128560

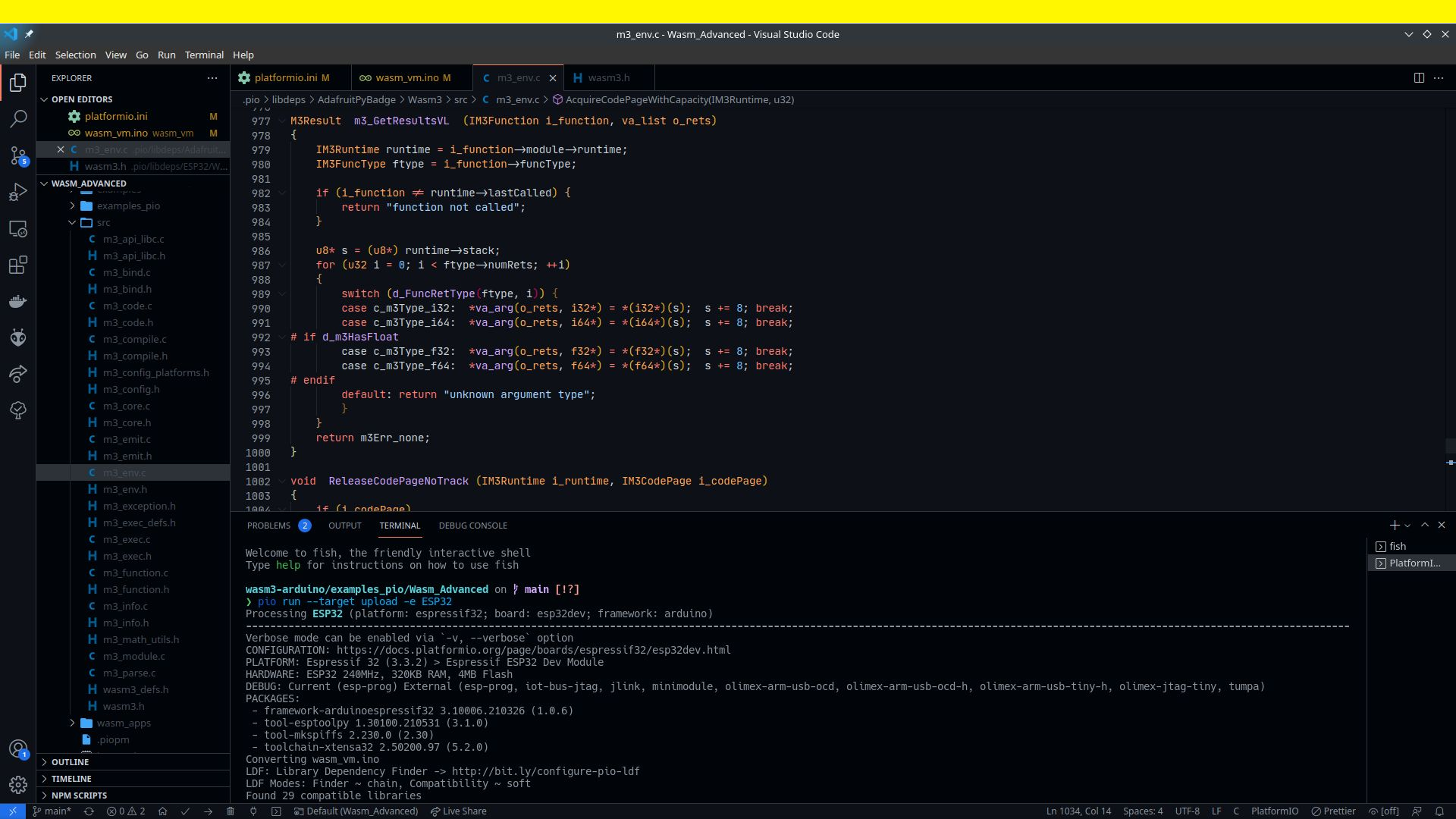
PlatformIO

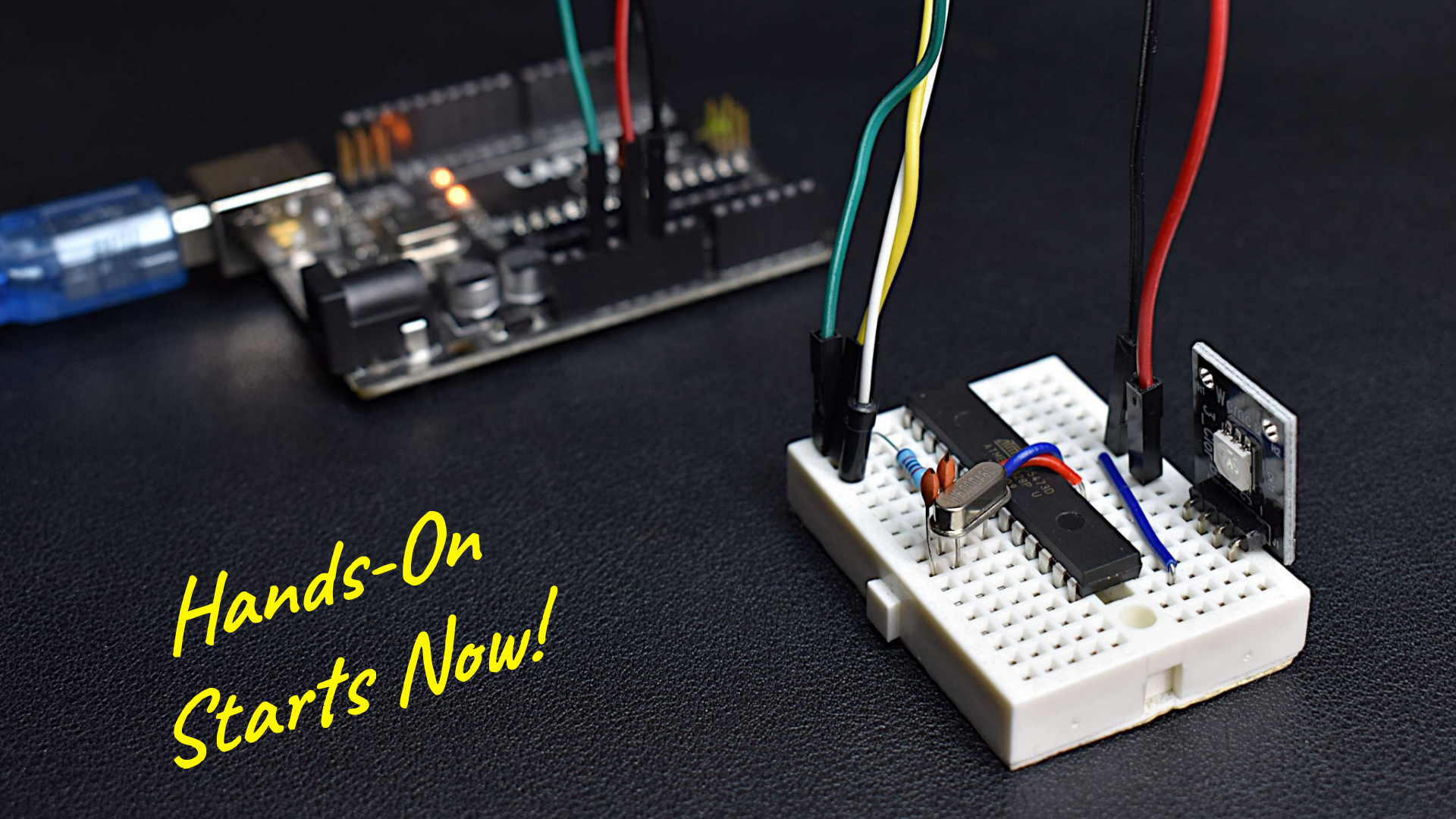
“**PlatformIO** is a **cross-platform, cross-architecture, multiple framework**, professional tool for embedded systems.”

- PlatformIO IDE, as a VS Code or Atom extension
- PlatformIO Core (CLI), standalone or as part of the extension
- Comes with:
  - Unit Testing
  - Static Code Analysis
  - Remote Development



PlatformIO, https://platformio.org/


m3_env.c - Wasm_Advanced - Visual Studio Code
File Edit Selection View Go Run Terminal Help
EXPLORER
OPEN EDITORS
platformio.ini M
wasm_vm.ino wasm_vm M
m3_env.c .pio/libdeps/AdafruitP...
wasm3.h .pio/libdeps/ESP32/W...
WASM_ADVANCED
examples_pio
src
m3_api_libc.c
m3_api_libc.h
m3_bind.c
m3_bind.h
m3_code.c
m3_code.h
m3_compile.c
m3_compile.h
m3_config_platforms.h
m3_config.h
m3_core.c
m3_core.h
m3_emit.c
m3_emit.h
m3_env.c
m3_env.h
m3_exception.h
m3_exec_defs.h
m3_exec.c
m3_exec.h
m3_function.c
m3_function.h
m3_info.c
m3_info.h
m3_math_utils.h
m3_module.c
m3_parse.c
wasm3_defs.h
wasm3.h
wasm_apps
.piopm
OUTLINE
TIMELINE
NPM SCRIPTS
platformio.ini M
wasm_vm.ino M
m3_env.c
wasm3.h
.pio > libdeps > AdafruitPyBadge > Wasm3 > src > m3_env.c > AcquireCodePageWithCapacity(IM3Runtime, u32)
977 M3Result m3_GetResultsVL (IM3Function i_function, va_list o_rets)
978 {
979 IM3Runtime runtime = i_function->module->runtime;
980 IM3FuncType ftype = i_function->funcType;
981
982 if (i_function != runtime->lastCalled) {
983 return "function not called";
984 }
985
986 u8* s = (u8*) runtime->stack;
987 for (u32 i = 0; i < ftype->numRets; ++i)
988 {
989 switch (d_FuncRetType(ftype, i)) {
990 case c_m3Type_i32: *va_arg(o_rets, i32*) = *(i32*)(s); s += 8; break;
991 case c_m3Type_i64: *va_arg(o_rets, i64*) = *(i64*)(s); s += 8; break;
992 # if d_m3HasFloat
993 case c_m3Type_f32: *va_arg(o_rets, f32*) = *(f32*)(s); s += 8; break;
994 case c_m3Type_f64: *va_arg(o_rets, f64*) = *(f64*)(s); s += 8; break;
995 # endif
996 default: return "unknown argument type";
997 }
998 }
999 return m3Err_none;
1000 }
1001
1002 void ReleaseCodePageNoTrack (IM3Runtime i_runtime, IM3CodePage i_codePage)
1003 {
PROBLEMS 2
OUTPUT
TERMINAL
DEBUG CONSOLE
Welcome to fish, the friendly interactive shell
Type help for instructions on how to use fish
wasm3-arduino/examples_pio/Wasm_Advanced on main [!?]
pio run --target upload -e ESP32
Processing ESP32 (platform: espressif32; board: esp32dev; framework: arduino)
Verbose mode can be enabled via `-v, --verbose` option
CONFIGURATION: https://docs.platformio.org/page/boards/espressif32/esp32dev.html
PLATFORM: Espressif 32 (3.3.2) > Espressif ESP32 Dev Module
HARDWARE: ESP32 240MHz, 320KB RAM, 4MB Flash
DEBUG: Current (esp-prog) External (esp-prog, iot-bus-jtag, jlink, minimodule, olimex-arm-usb-ocd, olimex-arm-usb-ocd-h, olimex-arm-usb-tiny-h, olimex-jtag-tiny, tumpa)
PACKAGES:
- framework-arduinoespressif32 3.10006.210326 (1.0.6)
- tool-esptoolpy 1.30100.210531 (3.1.0)
- tool-mkspiffs 2.230.0 (2.30)
- toolchain-xtensa32 2.50200.97 (5.2.0)
Converting wasm_vm.ino
LDF: Library Dependency Finder -> http://bit.ly/configure-pio-ldf
LDF Modes: Finder ~ chain, Compatibility ~ soft
Found 29 compatible libraries
fish
PlatformI...
main*
0 2
Default (Wasm_Advanced)
Live Share
Ln 1034, Col 14
Spaces: 4
UTF-8
LF
C
PlatformIO
Prettier
[off]


Hands-On
Starts Now!

platformio.ini - Project Configuration File

```
[platformio]
default_envs = d1_mini
```

Useful for more than one target

```
[env:d1_mini]
platform = espressif8266
board = d1_mini
framework = arduino
monitor_speed = 115200
```

Target microcontroller

Target board

Target framework / OS

Serial Port baudrate

https://docs.platformio.org/en/latest/projectconf/index.html

~~Hello World~~ Blink (src/main.ino)

```
// the setup function runs once when you press reset or power the board

#include <Arduino.h>

#define LED D4

void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED, OUTPUT);
}

// the loop function runs over and over again forever

void loop() {
  digitalWrite(LED, HIGH);           // Arduino: turn the LED on (HIGH)
                                     // D1 Mini: turns the LED *off*
  delay(1000);                       // wait for a second
  digitalWrite(LED, LOW);            // Arduino: turn the LED off (LOW)
                                     // D1 Mini: turns the LED *on*
  delay(1000);                       // wait for a second
}
```

Pull up vs pull down resistors;
https://www.seeedstudio.com/blog/2020/02/21/pull-up-resistor-vs-pull-down-differences-arduino-guide/

https://www.arduino.cc/reference/en/#structure

Blink & Hello from Serial World (src/main.ino)




```
#include <Arduino.h>

#define LED D4

void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED, OUTPUT);
  // initialize serial output
  Serial.begin(115200);
}

void loop() {
  digitalWrite(LED, HIGH);           // Arduino: turn the LED on (HIGH)
                                     // D1 Mini: turns the LED *off*
  Serial.println("Hello ...");       // Prints Hello to Serial
  delay(1000);                       // wait for a second
  digitalWrite(LED, LOW);            // Arduino: turn the LED off (LOW)
                                     // D1 Mini: turns the LED *on*
  Serial.println(" ... World!");     // Prints World! to Serial
  delay(1000);                       // wait for a second
}
```

https://www.arduino.cc/reference/en/#functions

The easy way to Interact with a Sensor

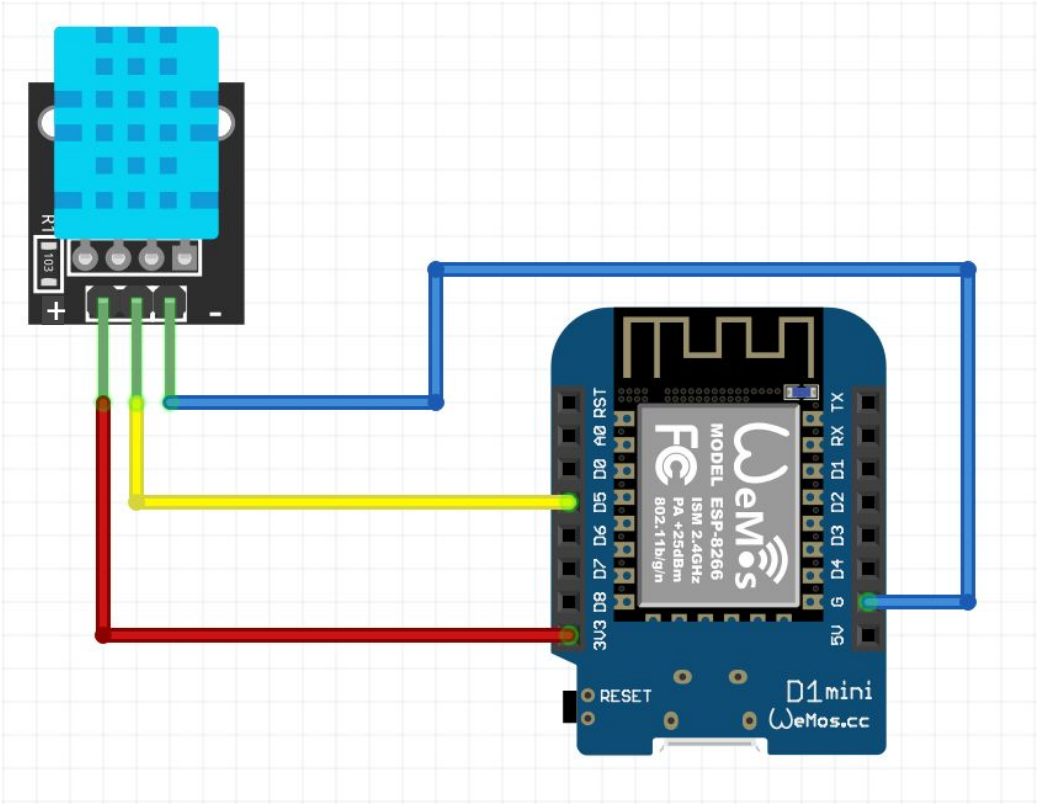



```
$ pio lib install "beegee-tokyo/DHT sensor library for ESPx"

# platform.ini is automatically updated with lib_deps (libs can be added manually to the file)
# lib_deps = beegee-tokyo/DHT sensor library for ESPx@^1.18.0
```

PIO Registry, https://docs.platformio.org/en/latest/projectconf/index.html

Wemos & DHT11 Circuit





https://www.etsy.com/shop/UnsafeWarnings

Reading the Data (src/main.ino)

```
#include "DHTesp.h"
⋮
#define DHTPIN D5
DHTesp dht;
⋮
void setup() {
  ⋮
  dht.setup(DHTPIN, DHTesp::DHT11); // connect DHT sensor to GPIO D5, and declare sensor type (DHT11)
}

void loop() {
  ⋮
  //delay(dht.getMinimumSamplingPeriod()); // this is not need if we main the 1000 delay

  float humidity = dht.getHumidity();
  float temperature = dht.getTemperature();

  Serial.printf("Temperature: %f, Humidity: %f\%\n", temperature, humidi
}
```

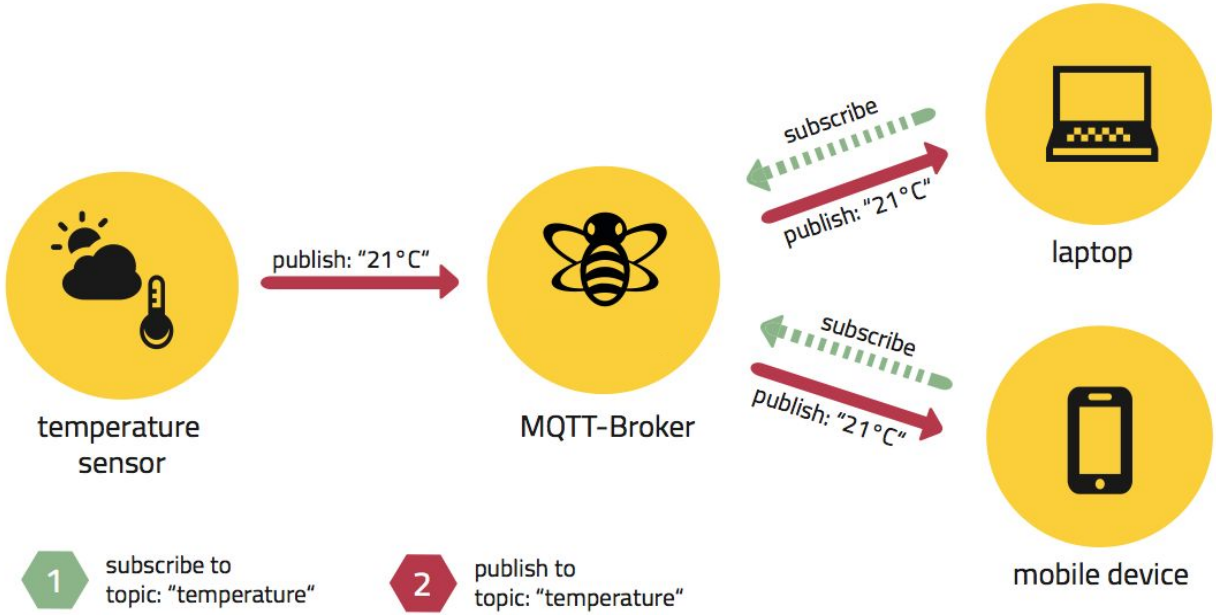



Break

The *I* in *I*oT stands for ~~*Security*~~ Internet…

but where is it?

MQTT and the world of Pub/Sub



QoS Levels:

- *At most once (0)*
- *At least once (1)*
- *Exactly once (2)*

Birth and Last Will and Testament (LWT) messages.

Birth *is used to send a message after the service has started, and the* **LWT** *is used to notify other clients about a disconnected client.*

TCP-based, can be used directly or with Web Sockets.

A little more on MQTT...

A MQTT broker is required, but there are several freely available, e.g.:

- Broker: **broker.emqx.io**
- TCP Port: **1883**
- Websocket Port: **8083**

To make it easy to experiment with, we will use MQTT over WebSockets.

- We can use the browser to interact with the broker without additional stuff.
- http://tools.emqx.io/

Side-quest: Wi-Fi

```
⋮
#include <ESP8266WiFi.h>
#include <WiFiClient.h>

const char *ssid = ".......";
const char *password = ".......";

void setup() {
  ⋮
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

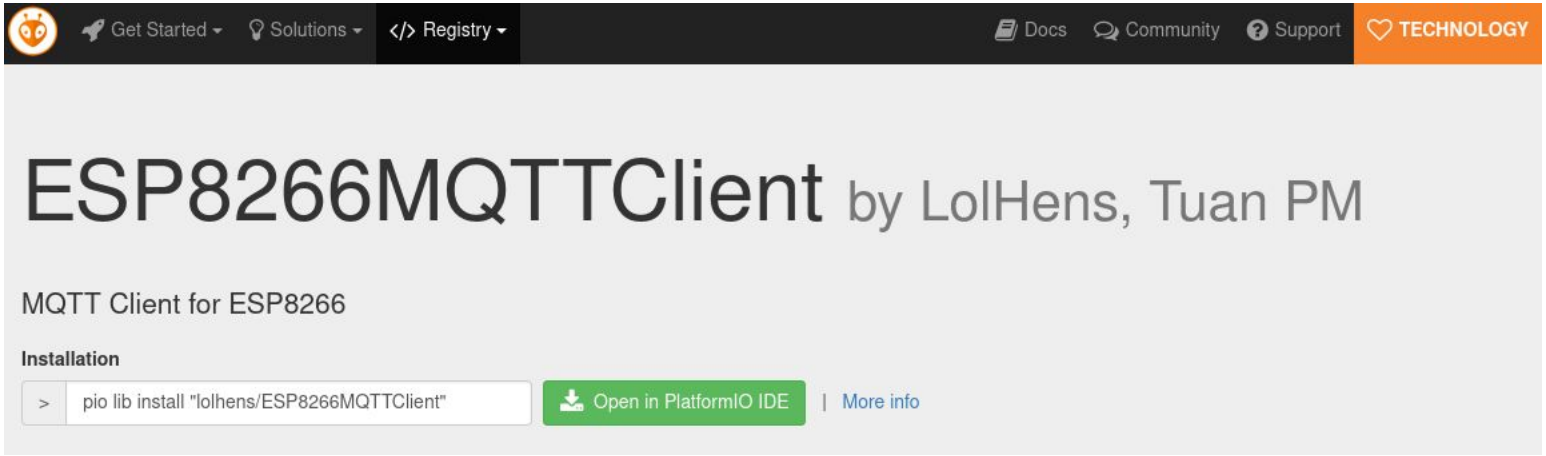
  Serial.println("Success!");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  ⋮
}
```



https://arduino-esp8266.readthedocs.io/en/3.0.2/

Getting the Libs




```
$ pio lib install "lolhens/ESP8266MQTTClient"
```

```
# lib_deps =
        beegee-tokyo/DHT sensor library for ESPx@^1.18.0
        lolhens/ESP8266MQTTClient @ ^1.1.1
```

PIO Registry, https://docs.platformio.org/en/latest/projectconf/index.html

MQTT *to the Internet and beyond!* (1/2)

```
⋮
#include <Hash.h>
#include <ESP8266MQTTClient.h>

MQTTClient mqtt;
const char *mqttBroker = "ws://broker.emqx.io:8083/mqtt";

void setup() {
  ⋮
  configTime(3 * 3600, 0, "pool.ntp.org", "time.nist.gov");

  ⋮ more on the next slide

  mqtt.begin(mqttBroker);
}

void loop() {
  ⋮
  mqtt.handle();
  mqtt.publish("/workshop123/temperature", String(temperature, 2), 0, 0);
}
```

We could use the secure version, but let's keep it unsafe for simplicity purposes.

There is no RTC on Wemos, thus we need to sync time on every boot.

```
//mqtt.begin(mqttBroker, {
      .lwtTopic = "workshop123/lwt",
      .lwtMsg = "offline",
      .lwtQos = 0,
      .lwtRetain = 0});
```

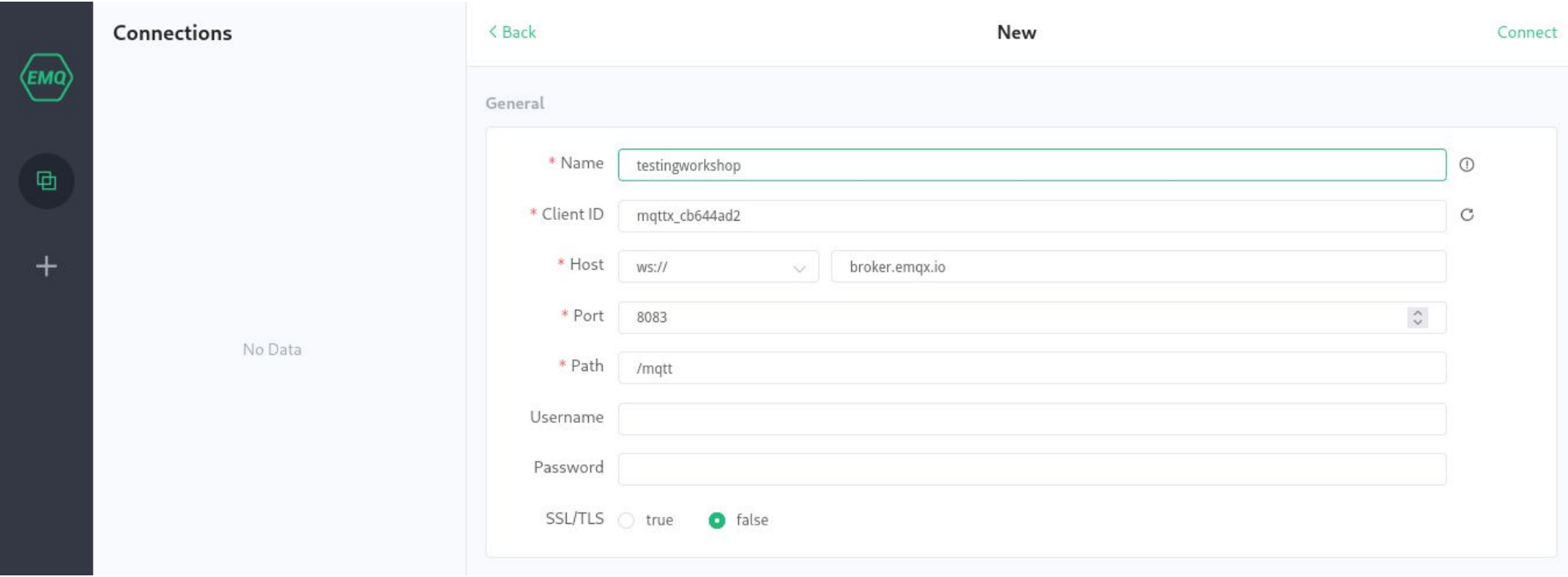
MQTT *to the Internet and beyond!* (2/2)

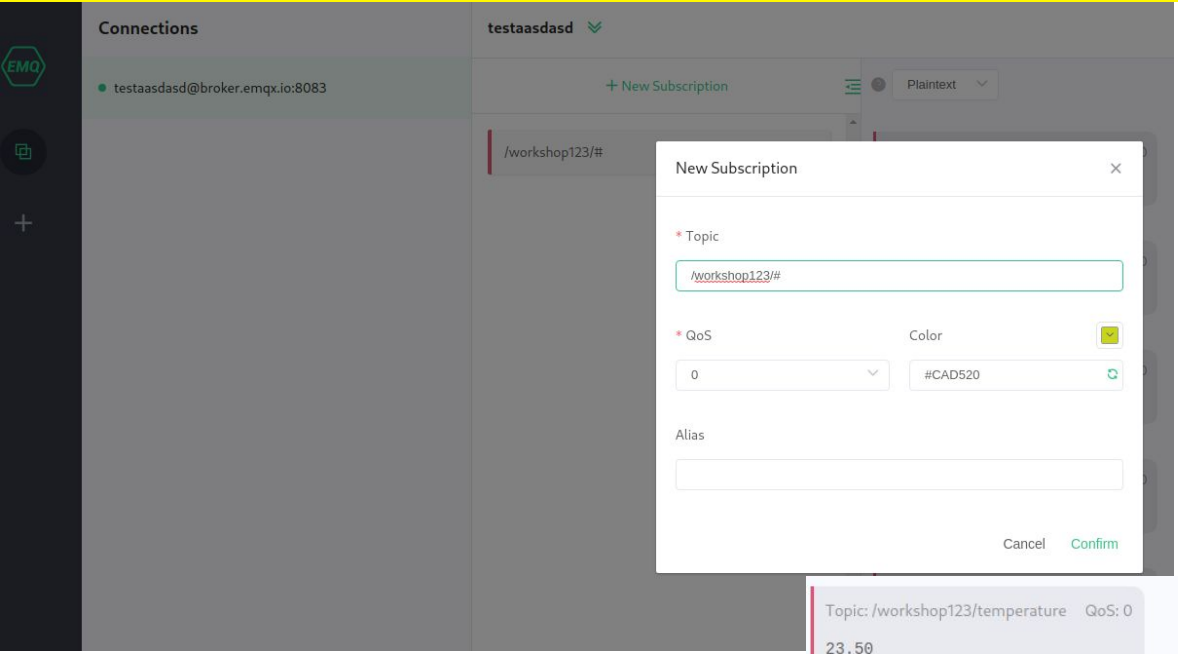
```
mqtt.onData([](String topic, String data, bool cont) {
  Serial.printf("Data rx, topic: %s, data: %s\n", topic.c_str(), data.c_str());
});

mqtt.onSubscribe([](int sub_id) {
  Serial.printf("Subscribe topic id: %d ok\n", sub_id);
});

mqtt.onConnect([]() {
  Serial.printf("MQTT: Connected\n");
  mqtt.subscribe("/workshop123/example", 1);
});
```

Interacting over Web (tools.emqx.io)





Topic: /workshop123/temperature QoS: 0

```
23.50
```

2021-10-21 19:56:48

Topic: /workshop123/example QoS: 0

```
{
  "msg": "hello"
}
```

2021-10-21 19:56:49

Topic: /workshop123/example QoS: 0

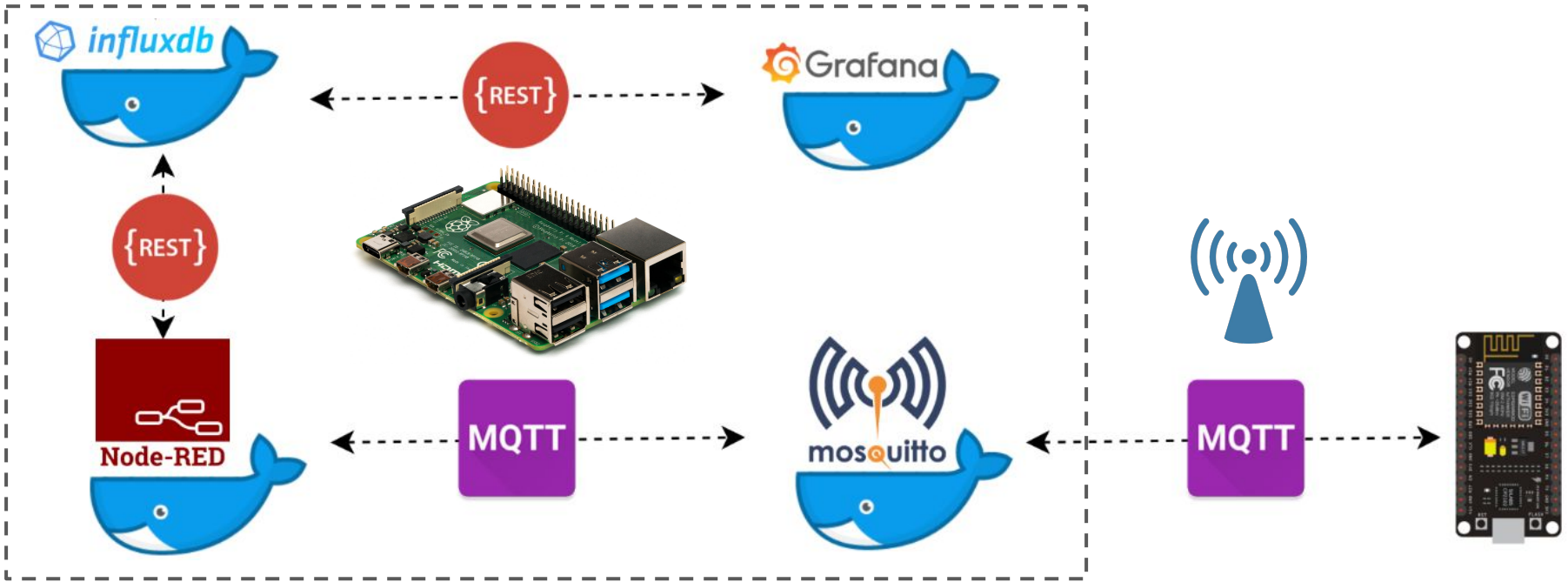
```
{
  "msg": "hello"
}
```

2021-10-21 19:56:49

Next steps (ideas)

- ➔ Publish JSON messages
  - ◆ Find a lib, install, read the example, ...
- ➔ Toggle the LED remotely
  - ◆ Subscribe and change state in accordance to the message.
- ➔ Program your system with Node-RED
  - ◆ Install it and make your first flow to periodically toggle the LED
- ➔ Make a Dashboard with Grafana
- ➔ Store historical data with InfluxDB or other Time-Series database
- ➔ Install and configure your own broker, dashboard and database
  - ◆ Mosquitto, InfluxDB, Grafana, and Node-RED in Docker

Motivational Example: *PiHeadquarters*



PiHeadquarters, https://github.com/jpdias/piheadquarters

Read More

- IoT for Beginners - A Curriculum, https://github.com/microsoft/IoT-For-Beginners
- OWASP Internet of Things (Top 10), https://owasp.org/www-project-internet-of-things/
- Build Computer from Scratch, https://eater.net/
- Adafruit Learning System, https://learn.adafruit.com/
- Pimoroni Learning, https://learn.pimoroni.com/
- Awesome IoT List, https://github.com/phodal/awesome-iot
- https://twitter.com/internetofshit
- Andreas Spiess, https://www.youtube.com/channel/UCu7_D0o48KbfhpEohoP7YSQ

Project ideas:

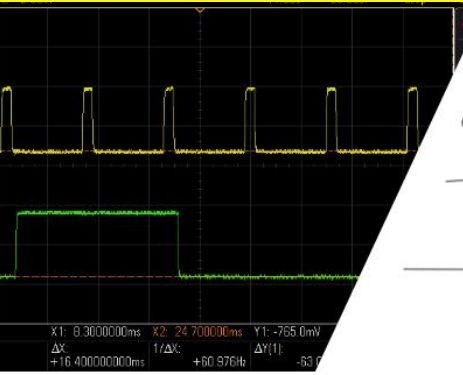
- https://hackster.io
- https://hackaday.com/
- https://create.arduino.cc/projecthub

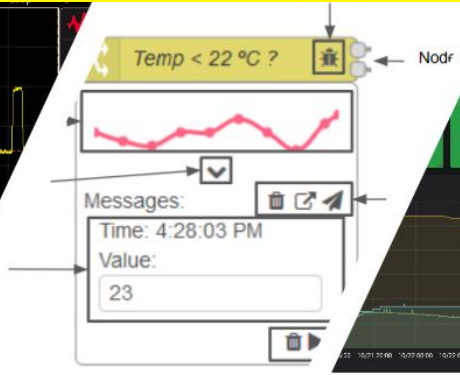
I want to spend *some* money...

$ AliExpress, all the components, cheap (pick 10-day delivery to ensure delivery)
$ PCBWay, https://www.pcbway.com/ (making PCB, 5 for 5$ + ports)

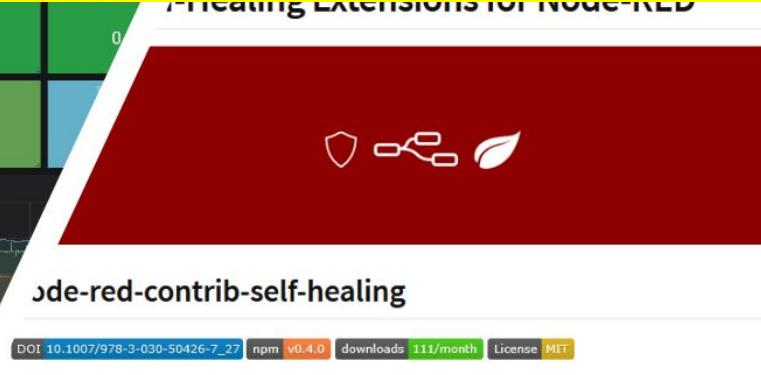
$$ Mauser.pt, https://mauser.pt/
$$ PTRobotics, https://ptrobotics.com/
$$ Mouser.com, https://pt.mouser.com/ (all the things, free ports +50€)
$$ Farnell.com, https://pt.farnell.com/ (all the things)

$$$ Pimoroni, https://shop.pimoroni.com/
$$$ Adafruit, https://www.adafruit.com/





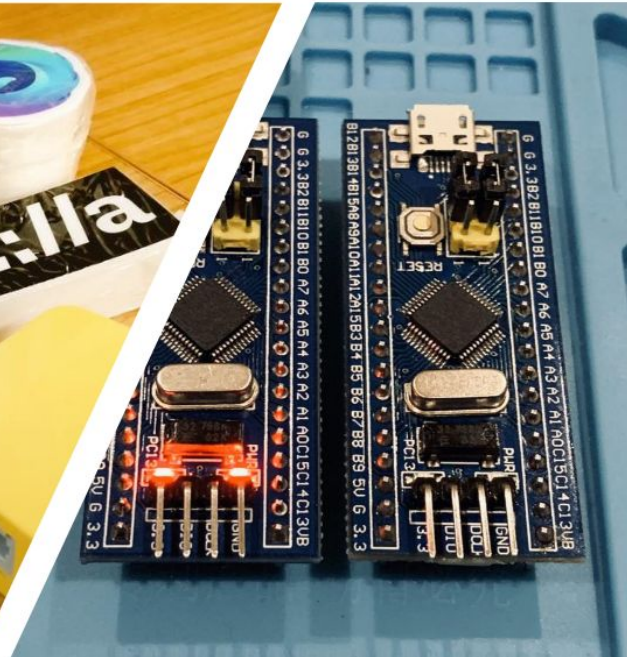




Call for Interest in IoT research:

- Software Engineering
- Visual programming & low-code
- Orchestration heterogeneous systems
- Autonomic Computing (self-healing)
- Fault-tolerance & Dependability
- Privacy & security
- Embedded and retro computing





It's a Wrap!

João Pedro Dias
https://jpdias.me
@jpd1as

Bruno Lima
https://brunolima.info

